FindMe

FindMe

Una bacheca di oggetti smarriti e ritrovati, aperta a tutti

Gioacchino (PM) · Micaela (Backend) · Giusy (Frontend) · Danilo (Tester)

Il progetto: tutto visibile, si interagisce con un account

1

Ospite

Apre il link e legge tutto: elenco, ricerca, filtri, dettagli e commenti.

2

Utente registrato

Dopo login o registrazione commenta, pubblica con il + e modifica i propri annunci.

3

Gli annunci

Oggetti smarriti e ritrovati, con categoria, luogo, data e stato: aperto o risolto.

Prototipo funzionante: il backend in Python viene collegato al frontend.

Quattro ruoli, una sola squadra

PM

Gioacchino

Pianifica le attività, coordina il gruppo e tiene la board.

BACKEND

Micaela

Python, logica del programma e database SQLite.

FRONTEND

Giusy

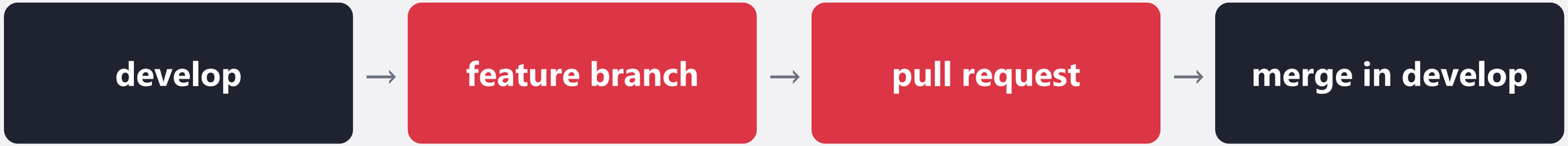
Pagine web, login, registrazione e interfaccia.

TESTER

Danilo

Prove manuali con dati di test per trovare i problemi.

Metodo (1): Agile con feature branch su GitHub



Cosa ha funzionato

- Un branch per ogni funzione, per ruolo
- Board pubblica con le attività
- Merge su develop con pull request

Cosa miglioriamo

- Le regole non sempre sono state rispettate
- Più disciplina su pull request e board
- Aggiornare i task man mano

Metodo (2): nuova repository, metodo diverso

Prima: repository iniziale

- Agile con develop e feature branch
- L'inserimento di Flask ha creato problemi
- Flask rimosso e progetto da adeguare

→

Dopo: nuova repository FindMe

- Stesso progetto, riproposto da zero
- File inseriti e collegati direttamente
- Lavoro diretto su main, senza develop

! **Siamo consapevoli dei rischi:** nessuna revisione con pull request, un errore finisce subito su main e modifiche agli stessi file possono andare in conflitto.

Backend (Micaela): dal menù al database

1

`main.py`

Menù in un ciclo while True con switch

→

2

`logic.py`

Funzioni richiamate da ogni opzione

→

3

`storage.py`

Connessione e persistenza sul DB

→

4

`SQLite`

Database con schema in infrastructure/

Come funziona: l'utente sceglie un'opzione dal menù, main.py richiama la funzione corrispondente in logic.py, e le funzioni di logic.py interagiscono con storage.py per leggere e scrivere sul database.

Frontend (Giusy): lettura libera, azioni da loggati

Da ospite (sola lettura)

- Elenco di tutti gli annunci
- Ricerca e filtri per tipo, categoria, stato
- Dettaglio e commenti

Da loggato

- Commentare gli annunci
- Pulsante + per aggiungere un annuncio
- Modificare ed eliminare i propri annunci
- Importare dati da un file JSON



3 pagine: index.html, login.html, register.html · HTML, CSS e JavaScript

Test e prossimi passi

Test (Danilo)

Prove manuali con dati di test: 10 annunci e 2 utenti.

- Login corretto e con dati errati
- Registrazione di un nuovo utente
- Permessi: ospite e utente
- Importazione di un JSON non valido

Prossimi passi

- Collegare frontend e backend con scambio JSON/CSV
- Esportare un risultato elaborato in un file separato
- Aggiungere test automatici
- Sostituire il salvataggio nel browser con il database